

Trabajo Practico N°8
de
Programación II

Estudiante: Tobias Leiva

Unidad: Interfaces y Excepciones

Universidad Tecnológica Nacional

Caso Practico

Parte 1: Interfaces en un sistema de E-commerce

1. Crear una interfaz Pagable con el método calcularTotal().
2. Clase Producto: tiene nombre y precio, implementa Pagable.
3. Clase Pedido: tiene una lista de productos, implementa Pagable y calcula el total del pedido.
4. Ampliar con interfaces Pago y PagoConDescuento para distintos medios de pago (TarjetaCredito, PayPal), con métodos procesarPago(double) y aplicarDescuento(double).
5. Crear una interfaz Notificable para notificar cambios de estado. La clase Cliente implementa dicha interfaz y Pedido debe notificarlo al cambiar de estado.

```
1. public interface Pagable {  
    public double calcularTotal();  
}  
  
2. public class Producto implements Pagable{  
    private String nombre;  
    private double precio;  
  
    public Producto(String nombre, double precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public double getPrecio() {  
        return precio;  
    }  
}
```

```

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    @Override
    public double calcularTotal() {
        return this.precio;
    }
}

3. public class Pedido implements Pagable {
    ArrayList<Producto> productos;
    private String estado;
    private Cliente cliente;

    public Pedido(String estado, Cliente cliente) {
        this.estado = estado;
        this.cliente = cliente;
        this.productos = new ArrayList();
    }

    public void agregarProducto(Producto producto) {
        productos.add(producto);
    }

    @Override
    public double calcularTotal() {
        double total = 0;
        for (Producto p : productos) {
            total += p.getPrecio();
        }
    }
}

```

```

        return total;
    }

    public void notificarEstado(String nuevoestado) {
        this.estado = nuevoestado;
        cliente.notificarCamboDeEstado(nuevoestado);
    }
}

4. public interface Pago {
    public void procesarPago(double monto);
}

public interface PagoConDescuento extends Pago{
    public double aplicarDescuento(double monto);
}

public class PagoConPayPal implements PagoConDescuento{

    @Override
    public double aplicarDescuento(double monto) {
        return monto - (monto * 0.15);
    }

    @Override
    public void procesarPago(double monto) {
        double total = aplicarDescuento(monto);
        System.out.println("El total con descuento es: " + total);
    }
}

public class PagoConTarjeta implements Pago{

    @Override
    public void procesarPago(double monto) {
        System.out.println("El total es: " + monto);
    }
}

```

```

    }

}

5. public interface Notificable {
    public void notificarCamboDeEstado(String nuevoEstado);
}

public class Cliente implements Notificable{

    private String nombre;

    public Cliente(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public void notificarCamboDeEstado(String nuevoEstado) {
        System.out.println(nombre + " tu pedido cambio de estado a " +
nuevoEstado);
    }
}

```

Parte 2: Ejercicios sobre Excepciones

1. División segura

- o Solicitar dos números y dividirlos. Manejar ArithmeticException si el divisor es cero.

```

public class DivisionSegura {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        double num1, num2;

        System.out.println("Ingrese dos numeros");
        System.out.println("Numero uno: ");
        num1 = scan.nextDouble();
    }
}

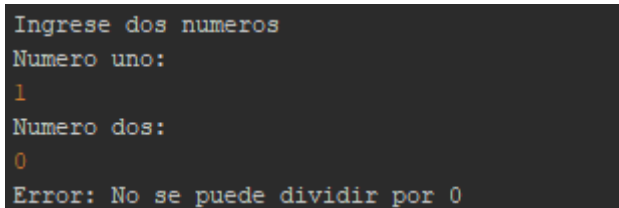
```

```

        System.out.println("Numero dos:");
        num2 = scan.nextDouble();

        try {
            if (num2 == 0) {
                throw new ArithmeticException("No se puede dividir por
0");
            }
            double resultado = num1 / num2;
            System.out.println("El resultado de la división es: " +
resultado);
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```



```

Ingrese dos numeros
Numero uno:
1
Numero dos:
0
Error: No se puede dividir por 0

```

2. Conversión de cadena a número

o Leer texto del usuario e intentar convertirlo a int. Manejar NumberFormatException si no es válido.

```

public class ConversionNumeroACadena {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        System.out.println("Ingrese un texto que pueda ser convertido
a entero");
        String textoAConvertir = scan.nextLine();

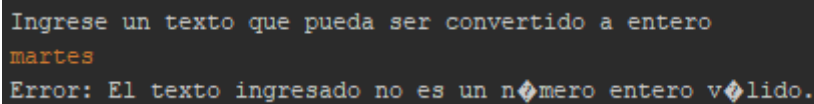
        try {
            int textoConvertido = Integer.parseInt(textoAConvertir);

```

```

        System.out.println("El número convertido es: " +
textoConvertido);
    } catch (NumberFormatException ex) {
        System.out.println("Error: El texto ingresado no es un
número entero válido.");
    }
}
}
}

```



```

Ingrese un texto que pueda ser convertido a entero
martes
Error: El texto ingresado no es un número entero válido.

```

3. Lectura de archivo

o Leer un archivo de texto y mostrarlo. Manejar FileNotFoundException si el archivo no existe.

```

public class LecturaArchivo {

    public static void main(String[] args) throws IOException {
        Scanner scan = new Scanner(System.in);

        System.out.println("Ingrese el nombre del archivo txt: ");
        String nombre = scan.nextLine();

        try {
            File archivo = new File(nombre);
            BufferedReader br = new BufferedReader(new
FileReader(archivo));
            System.out.println(br.readLine());
        } catch (FileNotFoundException e) {
            System.out.println("Error: El archivo no fue
encontrado.");
        } catch (IOException ex){
            System.out.println("Error de E/S");
            System.out.println(ex.getMessage());
        }
    }
}

```

```
}
```

```
Ingrese el nombre del archivo txt:
texto
Error: El archivo no fue encontrado.
```

4. Excepción personalizada

- o Crear EdadInvalidaException. Lanzarla si la edad es menor a 0 o mayor a 120. Capturarla y mostrar mensaje.

```
public class ExcepcionPersonalizada {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        System.out.println("Ingrese una edad");
        int edad = Integer.parseInt(scan.nextLine());
        if (edad <= 0 || edad >= 120) {
            throw new EdadInvalidaException("Ingreso una edad
            invalida");
        }
    }
}
```

```
Ingrese una edad
125
Exception in thread "main" Excepciones.EdadInvalidaException: Ingreso una edad invalida
at Excepciones.ExcepcionPersonalizada.main(ExcepcionPersonalizada.java:13)
```

5. Uso de try-with-resources

- o Leer un archivo con BufferedReader usando try-with-resources. Manejar IOException correctamente.

```
public class TryWithResources {

    public static void main(String[] args) {
        File archivo = new
        File("C:\\Users\\Lenovo\\OneDrive\\Documentos\\NetBeansProjects\\Ejerc
        itacionTpOcho\\src\\main\\java\\Excepciones\\ArchivoALeer.txt");

        try(BufferedReader br = new BufferedReader(new
        FileReader(archivo))){
            System.out.println(br.readLine());
        }
    }
}
```



```
    } catch(IOException ex) {  
        System.out.println("Error de E/S: "+ ex.getMessage());  
    }  
}  
}
```

```
Error de E/S: C:\Users\ASUS\OneDrive\Documentos\NetBeansProjects\Tp8\src\main\java\Excepciones\ArchivoALeer.txt (El sistema no puede encontrar la ruta especificada)  
-----  
BUILD SUCCESS
```

Repositorio: <https://github.com/Tobias-L7/Trabajos-Practicos-P2.git>